

GNN Guided Mapping of Data Flow Graphs for Coarse-Grained Reconfigurable Arrays

Capstone Project Final Report

Deniz Lopes Günes



Bachelor's in Informatics and Computing Engineering

U.Porto's Tutor: Professor João Cardoso
Company's Tutor: Professor Nuno Paulino

June 2026

Contents

Glossary	2
1 Introduction	4
1.1 Context	4
1.2 Technical concepts	4
1.3 Goals and expected results	6
1.4 Report structure	6
2 Methodology and main activities	6
2.1 Methodology	6
2.2 Stakeholders, roles and responsibilities	7
2.3 Activities carried out	7
2.3.1 First meeting (20 February 2026)	8
2.3.2 Second meeting (27 February 2026)	8
2.3.3 Third meeting (6 March 2026)	9
2.3.4 Fourth meeting (20 March 2026)	9
2.3.5 Fifth meeting (27 March 2026)	9
2.3.6 Sixth meeting (10 April 2026)	9
2.3.7 Seventh meeting (17 April 2026)	10
2.3.8 Eighth meeting (24 April 2026)	10
2.3.9 Ninth meeting (30 April 2026)	10
3 Development of the solution	11
3.1 Requirements	11
3.2 Architecture and technologies	11
3.2.1 DFG model	12
3.2.2 Synthetic graph generation	12
3.2.3 CGRA model	13
3.2.4 One-shot mapping validity	14
3.2.5 DFG partitioning	15
3.2.6 Ground-truth generation	15
3.2.7 Materialisation of inter-partition dependencies	17
3.2.8 GNN training	17
3.2.9 GNN-guided partitioning and repair	17
3.3 Developed solution	19
3.4 Validation	20
3.4.1 Experimental setup	20
3.4.2 Experimental results	21
4 Conclusions	22
4.1 Results achieved	22
4.2 Lessons learned	23
4.3 Future work	23
4.4 SDG impact	24

Glossary

AI *Artificial Intelligence*; field of computing dedicated to techniques that enable systems with adaptive or learned behaviour.

ALAP *As Late As Possible*; classical scheduling priority that schedules each operation at the latest cycle compatible with its data dependencies.

ALU *Arithmetic Logic Unit*; combinational unit inside a PE that performs the arithmetic and logical operations of the supported opcodes.

ASAP *As Soon As Possible*; classical scheduling priority that schedules each operation at the earliest cycle compatible with its data dependencies.

CGRA *Coarse-Grained Reconfigurable Architecture*; reconfigurable architecture composed of an array of configurable processing units.

CGRA-ME *CGRA Modelling and Exploration*; open-source CGRA design framework considered as an architecture reference in this project.

DAG *Directed Acyclic Graph*; directed graph with no cycles, used here as the underlying structure of a DFG.

DFG *Data Flow Graph*; directed graph that represents operations as nodes and data dependencies as edges.

DQN *Deep Q-Network*; reinforcement learning agent that approximates the Q-function with a deep neural network.

FEUP *Faculdade de Engenharia da Universidade do Porto*; Faculty of Engineering of the University of Porto.

GNN *Graph Neural Network*; neural network designed to process data structured as graphs.

INESC TEC *Instituto de Engenharia de Sistemas e Computadores, Tecnologia e Ciência*; Institute for Systems and Computer Engineering, Technology and Science.

JSON *JavaScript Object Notation*; text-based format used to store and exchange structured data.

KMS *Kernel Mobility Schedule*; modulo-scheduling construct that bounds the cycles in which each operation may be issued, used by SAT-MapIt [10] to reduce the SAT search space.

L.EEC *Licenciatura em Engenharia Eletrotécnica e de Computadores*; Bachelor's Degree in Electrical and Computer Engineering at FEUP.

- label0** Label predicted by the LISA-GNN and used as a priority in the greedy partitioner.
- LISA** *Graph Neural Network Based Portable Mapping on Spatial Accelerators* [8]; GNN-based approach used to guide mapping on spatial accelerators.
- LISA-GNN** GNN module used in the prototype, inspired by the LISA [8] approach, to predict priorities for DFG nodes.
- LLVM** Compiler infrastructure project providing modular and reusable compiler and toolchain technologies, used by the reference LISA flow.
- Mapper** Component that searches for or verifies a mapping solution.
- Mapping** Process of assigning the operations of a DFG to the physical resources of the CGRA.
- NaviMap** GNN plus deep-Q-network mapping approach for CGRAs, considered and ultimately discarded as a third leg of the prototype.
- One-shot** Model in which a partition of the DFG is mapped onto a single spatial configuration of the CGRA.
- PE** *Processing Element*; individual processing unit within the CGRA.
- SAT** *Boolean Satisfiability*; problem of determining whether an assignment exists that satisfies a set of Boolean constraints.
- SAT-MapIt** SAT-based mapper for CGRAs [10], used as a technical reference in this project.
- SDG** *Sustainable Development Goals*; United Nations Sustainable Development Goals.
- SMT** *Satisfiability Modulo Theories*; generalisation of SAT with theories such as arithmetic, arrays and bit-vectors.
- Z3** SMT/SAT solver [1] used to discharge the mapping constraints in the prototype.

1 Introduction

1.1 Context

This report documents the work carried out in the Capstone Project course unit, part of the Bachelor’s in Informatics and Computing Engineering at the Faculty of Engineering of the University of Porto (FEUP), during the second semester of the 2025/2026 academic year.

The project was developed at **INESC TEC**, under the supervision of Professor Nuno Paulino and the tutorship of Professor João Cardoso. INESC TEC is an associate laboratory active in engineering and computer science. The work was integrated in the *Optical and Electronic Technologies* area of the **CTM** centre (Centre for Telecommunications and Multimedia), which hosts applied research in compilation and reconfigurable architectures, the areas in which this project is positioned.

The work is integrated in **AIMACOV**, an ongoing national research project of the host team. Within that effort I was inserted in a working group of five colleagues, accompanied on by the project’s grant holders, who provided technical guidance throughout the semester.

The goal was to investigate and implement an approach inspired by LISA[8], based on *Graph Neural Networks* (GNNs), for the *mapping of Data Flow Graphs* (DFGs) onto *Coarse-Grained Reconfigurable Architectures* (CGRAs).

1.2 Technical concepts

Before going further, this subsection gives a quick overview of the main ideas behind the project, so the rest of the report can be read without jumping to external references.

CGRA. A *Coarse-Grained Reconfigurable Architecture* is a hardware accelerator made of a grid of small *Processing Elements* (PEs), usually arranged as a 2D mesh. Each PE is basically an ALU with a tiny register file, and PEs talk to their direct neighbours through a fixed interconnect. The difference from an FPGA is the granularity: FPGAs are reconfigured at the bit level, while CGRAs are reconfigured at the word level. Each cycle, every PE is told which operation to run and where to read its operands from. This makes CGRAs a good fit for compute-heavy kernels with regular data flow, because they give up some of the flexibility of an FPGA in exchange for better energy efficiency and much smaller configuration data.

Mapping process. *Mapping* is the compilation step that decides which PE runs each operation and, if cycles are also considered, when. The input is normally a *Data Flow Graph* (DFG): a directed acyclic graph where nodes are operations and edges are data dependencies. The mapper has to respect three

kinds of constraints: *operation compatibility* (a node can only go on a PE that supports its opcode), *connectivity* (if $u \rightarrow v$ is a dependency, the PEs assigned to u and v must be directly connected, or the value has to be routed through other PEs or memory), and *resource limits* (no PE runs more than one operation in the same cycle, fan-in/fan-out budgets are respected, and so on). Mapping is NP-hard in general, so the literature has explored both exact approaches (SAT/ILP-based) and heuristics.

Graph Neural Networks. A *Graph Neural Network* (GNN) is a type of neural network that works directly on graphs. Each node starts with a feature vector and, through several rounds of message passing, mixes in information from its neighbours. After a few rounds, each node ends up with an embedding that captures both its own features and the structure around it, which can then be used to predict per-node, per-edge or per-graph properties. GNNs fit DFG-based problems naturally, because the input is already a graph and the things one usually wants to predict (scheduling priorities, distances, placement hints) live on the nodes.

LISA. LISA[8] is a GNN-based approach for mapping DFGs onto spatial accelerators. Instead of trying to solve the mapping itself, the GNN is trained on traces from a reference mapper to predict a few labels per DFG node, such as scheduling order and spatial/temporal distances to dependent nodes. These labels are then handed to a downstream mapper as priorities or constraints, which nudges it towards solutions similar to the reference flow while staying portable across different accelerator targets.

SAT-MapIt. SAT-MapIt[10] is an exact, SAT-based mapper for CGRAs. It encodes the mapping problem as Boolean constraints over placement variables, and uses a *Kernel Mobility Schedule* (KMS), a modulo-scheduling construct that bounds the cycles in which each operation can be issued, to shrink the search space before the SAT solver kicks in. The output is either a correct mapping (placement plus modulo schedule) or a proof that no mapping exists for the given resource budget.

In this project, LISA and SAT-MapIt are the two main references that the prototype combines: a LISA-style GNN provides per-node priorities, and a simplified, SAT-MapIt-inspired mapper checks each one-shot partition built from those priorities.

Related approaches. LISA is not the only learned approach to CGRA mapping. GRAFT[7] predicts a placement distribution through GNN-based sub-graph matching to steer a progressive graph-based mapper, reporting a $64.5\times$ mapping speedup over LISA with comparable or better mapping quality on CGRAs up to 16×16 . Both LISA and GRAFT, however, assume the DFG fits within a single CGRA configuration and focus on accelerating placement and routing; neither addresses the partitioning problem that arises when this

assumption does not hold. When a DFG exceeds the capacity of a single configuration, traditional approaches apply generic graph-partitioning algorithms such as the Kernighan-Lin heuristic or multi-level frameworks such as METIS[6] to minimise edge cuts. These methods are agnostic to CGRA topology and routing constraints, so the partitions they produce are not guaranteed to be embeddable onto the target hardware. This project instead learns the partitioning step itself, predicting the partition structure prior to mapping.

1.3 Goals and expected results

The goals defined for the project were the following:

- Study the state of the art on *mapping* of DFGs to CGRAs, in particular GNN-based approaches inspired by LISA.
- Build a prototype that departs from current approaches by using GNNs to guide *mapping* decisions.
- Evaluate the solution with end-to-end examples and contrast it with a purely algorithmic strategy.

1.4 Report structure

The next section frames the methodology, stakeholders and activities. The development section describes the DFG model, the CGRA model, the partitioning, the materialisation of dependencies between partitions, the SAT validation and the integration of the LISA-GNN. The experimental validation, known limitations and future work follow.

2 Methodology and main activities

2.1 Methodology

This project was conducted at INESC TEC in a hybrid model. I had the flexibility to work on the project remotely and to communicate with my supervisors and the rest of the team through a Slack[9] workspace that belongs to the team and to which I was invited.

Throughout the process, weekly meetings were held in the INESC TEC room in which I presented what had been accomplished during the week and we discussed the results and what should be addressed in the following one. For these meetings I prepared a short presentation of about 5-10 minutes. These meetings were an opportunity not only to clarify questions about the project and its implementation, but also to receive feedback from the supervisors regarding the current state of the prototype and the features to be integrated next. This

is also why the project requirements evolved significantly over the semester: although the main objective was clear (use *AI* to guide the mapping process), the structure of the pipeline and exactly which state of the art approaches to follow were not fixed at the start, and the meetings were essential to decide which ideas were appropriate given the project’s goals and duration.

The development overall followed an iterative methodology[2], well suited to a research project of this nature. The project was developed inside a private GitLab[4] repository belonging to the team.

The technical resources used were:

- GitLab[4] for version control.
- Slack[9] for day-to-day communication with the team.

2.2 Stakeholders, roles and responsibilities

The project was developed individually by Deniz Lopes Günes.

- **Deniz Lopes Günes (author)**: literature review, implementation of the DFG generator, the partitioner, the integration of the SAT solver, the implementation of a SAT-MapIt-inspired one-shot mapper, the training and inference of the LISA-GNN, the end-to-end examples and the writing of the report.
- **Francisco Mouro Sampaio Nunes (team colleague, L.EEC)**: developed a project vertical to mine, responsible for generating the code used to configure the *CGR*A from the *mappings* produced.
- **Professor Nuno Paulino (main supervisor, INESC TEC)**: technical guidance, definition of goals, critical review of the work and validation of the results.
- **Guilherme Cruz (co-supervisor, INESC TEC)**: technical support and discussion of implementation strategies.
- **Henrique Teixeira (co-supervisor, INESC TEC)**: technical support and discussion of implementation strategies.
- **Professor João Cardoso (tutor, FEUP)**: academic accompaniment, alignment with the course unit’s requirements and critical review of the report.

2.3 Activities carried out

The technical activities can be grouped into the following blocks:

- **DFG representation**: definition of a JSON format for nodes, operations, constants, metadata and dependency edges.

- **Example generation:** creation of synthetic graphs inspired by matrix multiplication and convolution patterns.
- **Partitioning:** development of strategies to split graphs that do not fit in a single CGRA configuration.
- **Materialisation:** conversion of inter-partition dependencies into synthetic *load/store* nodes.
- **Mapping validation:** SAT verification of each materialised partition, respecting operation compatibility, connectivity between PEs and resource limits.
- **LISA-GNN integration:** use of *label0* predictions as a priority for node selection in the greedy partitioner.
- **Evaluation:** execution of end-to-end examples and collection of metrics on partitioning, cuts and SAT validation.

The initial phases (study, DFG generation, partitioning) took place between the start of the semester and mid-March; the LISA-GNN integration and the end-to-end pipeline were consolidated in May; the evaluation and report writing followed in the final weeks.

The remainder of this section walks through the project’s evolution meeting by meeting, which is the most faithful way to describe how the direction of the work was iteratively reshaped.

2.3.1 First meeting (20 February 2026)

The first meeting was used to re-introduce the team, agree on the communication channel (Slack) and align on the immediate reading material. I was tasked with surveying the state of the art on graph *mapping* for CGRAs, in particular GNN-based approaches, so that the next meeting could be grounded on concrete papers rather than on the generic proposal.

2.3.2 Second meeting (27 February 2026)

After the initial reading pass, the supervisors asked me to bring a dataset of DFGs of the kind used in the state-of-the-art papers, to review two or three of the most cited mappers, and to try out a conventional mapper as a baseline. The discussion also opened two questions that would stay with the project until the end: how to check that a *mapping* is valid, and how to represent a CGRA as a graph. The recommendation was to keep both representations as simple as possible.

2.3.3 Third meeting (6 March 2026)

By the third meeting I had cloned the LISA[8] DFG generator and produced a first synthetic dataset of graphs. Reproducing the LISA flow end-to-end ran into a practical blocker: LISA itself relies on *Gurobi* on top of an *LLVM* build, and the *LLVM* dependency could not be installed cleanly in the working environment. In parallel, I started reading SAT-MapIt[10], which uses a *Kernel Mobility Schedule* (KMS), a construct from *modulo scheduling* that bounds the cycles in which each operation may be issued, to reduce the SAT search space. The key idea that came out of this meeting was the one that ended up defining the whole project: rather than picking one of the two approaches, combine them. Use the LISA-style GNN to add constraints to the graph, and let a SAT-based mapper consume those constraints.

2.3.4 Fourth meeting (20 March 2026)

Two things were settled in this meeting. First, the LISA flow was reproduced for the homogeneous case: 1000 DFGs were generated for a 4×4 CGRA and another 1000 for an 8×8 CGRA, although mapping all of them with the reference flow turned out to be impractical (roughly 40 graphs mapped in a full day of runtime for the 4×4 case). Second, I started reading *NaviMap*, a GNN-plus-*deep-Q-network* mapping approach, and we discussed whether the LISA labels could be fed into a NaviMap-like agent. The idea was left open as a possible third leg of the prototype, to be evaluated in the following meeting after a deeper read of the paper.

2.3.5 Fifth meeting (27 March 2026)

This meeting was when the NaviMap direction was dropped. After a closer read, combining a *deep-Q-network* agent on top of the GNN labels turned out to add significant implementation and training complexity, having to handle different input shapes for the Neural Networks as well as a lack of open-source availability from NaviMap, without a clear advantage over the simpler LISA-plus-SAT-MapIt pairing for the scale of graphs we were targeting. The decision was therefore to set NaviMap aside and to concentrate the remaining effort on the LISA-style labels combined with the SAT mapper.

2.3.6 Sixth meeting (10 April 2026)

The discussion in this meeting was almost entirely about the *codegen* side of the project, led by Francisco. From my side, the main takeaway was that the mapping flow had to produce a clean, stable JSON output, so that the downstream code generation could consume it without further negotiation about the format.

2.3.7 Seventh meeting (17 April 2026)

At this point I had a Python CGRA class that is passed to the mapper, LISA was producing four labels per node (used as features), and the experimental SAT path was using two of those labels (temporal and spatial distance) on top of a *CGRA-ME*-style template to produce reference mappings for that prototype stage. The meeting surfaced a real, recurring problem: the DFG formats used by LISA and by SAT-MapIt did not agree. LISA worked with a small set of operations (`const`, `load`, `store`, `add`, `mul`, `sub`, ...), whereas SAT-MapIt also reasoned about branches, *live-ins/live-outs* and other entities. The action item was to consolidate a single input format and intermediate JSON, under the assumption that everything is one-shot, and to check whether SAT-MapIt could be coerced into a one-shot regime. This meeting was when we decided to simplify the problem by focusing on one-shot mapping with no temporal constraints.

2.3.8 Eighth meeting (24 April 2026)

This is the meeting in which the eventual shape of the prototype was decided. The problem on the table was that SAT-MapIt is built around *modulo scheduling*, its notion of an *initiation interval*, and the way it places operations across cycles all assume that regime, whereas LISA does not directly emit a SAT-MapIt KMS or a complete modulo schedule. It predicts labels such as schedule order and spatial/temporal distances that guide a downstream mapper. Plugging the LISA labels straight into SAT-MapIt was therefore not straightforward. Due to time constraints and the scope of the Capstone Project, the team decided to simplify and abstract the mapping into one-shot mapping. This would raise an issue, where the LISA labels would be pointless since all the nodes would be place at timestep 0. My suggestion was to invert the approach: take a large DFG, run a learned model on it to obtain per-node features, use those features to *split* the graph in such a way that cut dependencies are made explicit through synthetic memory operations, and then one-shot *map* each subgraph with the SAT mapper. The open question became how to perform the split itself, whether by a heuristic or by another learned component.

2.3.9 Ninth meeting (30 April 2026)

The ninth meeting confirmed that the partitioned one-shot strategy worked. A greedy partitioner was already in place, not yet driven by a GNN, but by hand-crafted priorities such as *ASAP*, *ALAP* and edge count — whose objective was to minimise the number of edges cut at the partition boundary, and thus the amount of synthetic memory traffic needed to keep memory operations outside of the CGRA’s ALU array. The splitting was working and each subgraph was being mapped one-shot with the SAT-MapIt-inspired one-shot mapper to produce reference labels for this abstraction. From here the plan was clean: use the original large graph, augmented with the per-partition timesteps as an extra feature, to train a GNN that predicts the schedule order (*label0*); then feed *label0* into the partitioner to obtain better partitions, and finally validate each

partition with the simplified SAT mapper. This is the flow that ended up being reported in this document.

3 Development of the solution

3.1 Requirements

The project was always very open-ended in terms of the technical solution, as long as it iterated on state of the art approaches using *AI*. One technical constraint was to agree on a JSON format for the output since another project for the codegen part was being developed in parallel, and the mapping output had to be clean and stable for that project to consume it without further negotiation.

The prototype does not cover cycle-by-cycle temporal scheduling. The SAT *mapper* only verifies the spatial validity of each partition, that is, it assumes that internal dependencies are resolved by direct connections between PEs in the same configuration.

3.2 Architecture and technologies

The primary goal of the approach is to achieve the high partition quality of exact mathematical solvers while maintaining the low computational overhead of greedy heuristics. The solution was implemented in Python and organised as a flow with two distinct phases, as shown in Figure 1: a training phase, in which a GNN learns optimal partition structure from exhaustively generated ground truth, and an inference phase, in which the trained GNN guides the partitioning of unseen DFGs with SAT-based spatial validation.

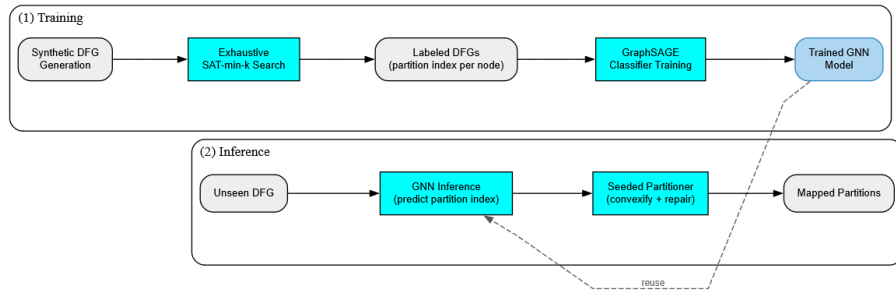


Figure 1: Overview of the proposed flow. The methodology is divided into two distinct phases: training the GNN, and inference-time graph partitioning with SAT-based spatial validation.

The flow comprises three stages: (1) synthetic graph generation, (2) ground-truth generation and GNN training, and (3) GNN-guided partitioning and spa-

tial mapping. Stages (1) and (2) constitute the GNN training phase, and stage (3) represents the DFG partitioning inference.

In summary, a GNN is trained to predict, for each instruction node of an input DFG, the partition it should belong to in an optimal split, that is, a split which minimises the inter-partition cut edges while maintaining spatial mappability on the target CGRA. To obtain a ground truth relating mapping quality to graph splits, an exhaustive SAT-MIN-K search is performed per training graph: every possible split is attempted and its mapping quality is recorded against the chosen split. The trained GNN then predicts, for an input test DFG, the best partition to which each node should belong, and adds this information as attributes on the nodes. A topology-aware partitioner processes the annotated graph, performing a final refinement to ensure mapping validity given the CGRA’s hardware constraints. By reducing the mapping search space to an initial inferred solution, the DFG mapping time is reduced. Each partition is finally verified by a SAT mapper implemented with Z3[1] and exported as a PE configuration.

3.2.1 DFG model

A DFG is modelled as a directed acyclic graph (DAG) in which the nodes represent operations and the edges represent data dependencies. Constants are root nodes and memory operations are represented explicitly through `load` and `store`. In the current model, all edges have zero distance, that is, a dependency inside a partition must be satisfied in the same one-shot configuration. The node types are drawn from a set of ten operations: arithmetic (`add`, `sub`, `mul`), bitwise logic (`and`, `or`, `xor`), shifts (`shl`, `shr`), and memory operations (`load`, `store`). Immediate constants are modelled as dedicated constant nodes with a value sampled uniformly from $[-256, 256]$.

3.2.2 Synthetic graph generation

Training a GNN-based mapper requires a large corpus of labelled DFGs that span a representative range of kernel shapes, data-dependency structures and memory-access patterns, while remaining within the resource bounds of the target CGRA. Because hand-written or compiler-extracted benchmarks are too few and too uniform for this purpose, a parametric generator was built that produces synthetic DFGs by starting from a small kernel template and applying a sequence of structure-preserving mutations.

Two loop-body templates seed the generator. The *matrix-multiplication* template encodes the inner accumulation $c += a[k] \times b[k]$ as a six-node DAG: two loads feeding a `mul`, whose result is added to a third load and then written back by a `store`. The *convolution* template encodes $y += w[k] \times (x[k] \ll 2)$ and extends this chain with a constant node and a `shl` operation, yielding an eight-node seed. Both templates are acyclic, represent one inner loop iteration, have exactly one memory write, and satisfy all CGRA structural invariants by construction.

To increase graph variability, the generator applies a sequence of graph mutations over the templates, partitioned into three structural classes:

- *Depth mutations* lengthen the critical path by inserting operations along existing producer-consumer chains, increasing pipeline depth.
- *Breadth mutations* introduce new parallel computation by branching live values into independent paths, merging independent values through binary operations, or adding fan-out to existing nodes without extending depth. These mutations increase ASAP-level width and routing pressure.
- *Neutral mutations* alter operations or operand types without changing graph topology, replacing nodes with others of the same type, or replacing a memory load with an immediate constant operand. These mutations increase operation-type diversity without affecting the graph’s structural complexity.

3.2.3 CGRA model

The CGRA is described by a grid of PEs. Each PE can support a subset of operations and, in some architectures, only some PEs can perform memory operations. The topology defines which PEs are directly connected. In the current experiments a homogeneous 4×4 CGRA was used. Figure 2 shows the canonical layout we assume: a 2D grid of PEs, with data and instruction memories on the side, where each PE is essentially an *ALU* with a local register file and can operate on the results delivered by its neighbouring PEs.

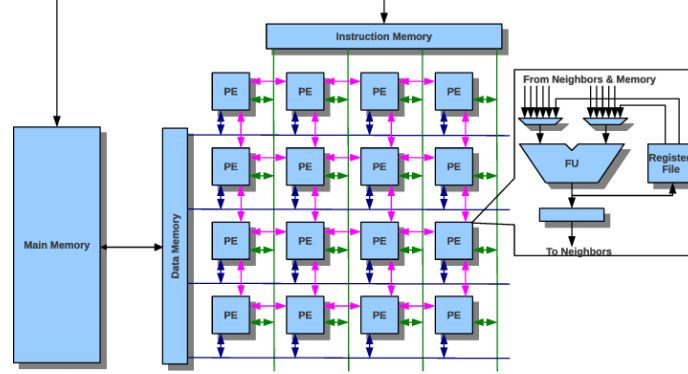


Figure 2: A 4×4 CGRA is essentially a 2D grid of PEs, plus data and instruction memories; each PE can operate on the results of its neighbouring PEs, is built around an *ALU* with a local register file, and shares a global storage area in the data memory reserved by the compiler. Figure reproduced from Jeyapaul *et al.*[5].

3.2.4 One-shot mapping validity

For a partition to be considered mappable, a custom SAT placement encoding, distinct from the modulo-scheduling formulation of SAT-MapIt[10], must find an assignment of nodes to PEs that satisfies the following conditions:

Let V_I be the set of instruction nodes of the partition, E_I the set of instruction-to-instruction edges, and $\mathcal{E}$ the set of PEs of the CGRA. For every $(v, e) \in V_I \times \mathcal{E}$ we introduce a Boolean placement variable

$$p_{v,e} \in \{0, 1\}, \quad p_{v,e} = 1 \iff \text{node } v \text{ is placed on PE } e.$$

The mapping is then encoded by the following clauses:

(C1) Operation compatibility. Let $op(v)$ denote the opcode of node v and $ops(e)$ the set of opcodes supported by PE e . For each node v and each PE e that does not support $op(v)$, the placement is forbidden:

$$\forall v \in V_I, \forall e \in \mathcal{E}, op(v) \notin ops(e) \implies \neg p_{v,e}. \quad (1)$$

(C2) Exactly one PE per instruction. Each instruction node is placed on exactly one PE, enforced as a pseudo-Boolean equality:

$$\forall v \in V_I, \sum_{e \in \mathcal{E}} p_{v,e} = 1. \quad (2)$$

(C3) At most one instruction per PE. Each PE may host at most one instruction of the partition:

$$\forall e \in \mathcal{E}, \sum_{v \in V_I} p_{v,e} \leq 1. \quad (3)$$

(C4) Same-cycle nearest-neighbour dependencies. Let $nbr(a, b)$ denote that PEs a and b are directly connected in the CGRA topology (or that $a = b$ when the model allows intra-PE forwarding). For every internal dependency $u \rightarrow v \in E_I$, at least one feasible neighbouring assignment must be selected:

$$\forall (u, v) \in E_I, \bigvee_{\substack{a, b \in \mathcal{E} \\ nbr(a, b)}} (p_{u,a} \wedge p_{v,b}). \quad (4)$$

This is a simplification of SAT-MapIt's temporal forward-edge constraint: dependent instructions may share the same partition (and therefore the same one-shot cycle) as long as their PEs are physically adjacent.

(C5) Memory-capable PEs. For each memory node $v \in N_{mem}$ (where $N_{mem} = V_{load} \cup V_{store}$) and each PE e that does not provide memory access ($e \notin mem$):

$$\forall v \in N_{mem}, \forall e \in \mathcal{E}, e \notin mem \implies \neg p_{v,e}. \quad (5)$$

On the homogeneous target evaluated here every PE supports every operation and every PE has data-memory access, so $mem = \mathcal{E}$ and (C5) is inactive; it is retained for heterogeneous architectures with a memory-capable PE subset.

Post-solution resource checks. Fan-in, fan-out and physical-link capacity are not encoded in the SAT formulation. Once Z3 returns a model, the prototype reads back the placements and rejects any solution in which, for the chosen limits, a PE exceeds its outgoing or incoming edge budget, or a physical link between two PEs is used by more edges than its capacity. A violation here is reported back to the caller as an infeasible mapping rather than as a SAT failure.

A structure such as $A \rightarrow C \leftarrow B$ is valid within a single partition if A , B and C are placed on compatible PEs and if the PEs of A and B can deliver their results directly to the PE of C , within the limits of C 's inputs and the physical links. It should be noted that the prototype covers spatial feasibility inside the partition, not temporal scheduling.

3.2.5 DFG partitioning

When the input finite DFG does not fit in a single CGRA configuration, the graph is split into a sequence of partitions $P_1, \dots, P_K$. Because partitions execute sequentially, a value can only be read from an equal-or-earlier partition, so to preserve the order of the dependencies the partitioner enforces:

$$partition(src) \leq partition(dest) \quad (6)$$

for each edge $src \rightarrow dest$. This constraint prevents dependencies that would have to flow back to a previous partition. Only such *monotone* assignments, which respect the topological order of the graph, are considered.

3.2.6 Ground-truth generation

After generating a set of DFGs, a training dataset for the GNN is built by systematic partition and mapping to valid solutions. A valid solution is defined by partitioning the DFG nodes into K non-empty subsets. When edges are cut between these subsets they are transformed into synthetic **load/store** pairs, and each resulting subset must then be successfully mapped.

The objective of generating ground truth is to find the partition assignment σ^* that jointly minimises the number of partitions K and the number of cut edges, subject to CGRA constraints. Once σ^* is found, each instruction node $v \in V_I$ is labelled with its assigned partition index $\sigma^*(v)$, which serves as the ground-truth target for the GNN's multi-class classification.

To find this assignment, an exhaustive SAT-MIN-K search (Algorithm 1) is performed over each synthetic DFG. The algorithm performs an upper-bounded sweep over K , starting from a resource-derived lower bound $K_{\min}$ (the instruction count $|V_I|$ divided by the number of PEs) up to $|V_I|$, the trivial case in which each partition contains a single instruction. For each value of K , all valid monotone assignments are enumerated exhaustively and SAT-checked, guaranteeing that the returned partition is minimal in K and optimal in edge cuts among all feasible K -partition solutions.

Each feasible partition is scored by the tuple:

$$\mathcal{S}(\mathcal{P}) = (|\text{cut}(\mathcal{P})|, |V_{\text{syn}}|, \max_i f_i, \max_i \rho_i, \max_i \ell_i) \quad (7)$$

where $|\text{cut}(\mathcal{P})|$ is the number of cut edges, $|V_{\text{syn}}|$ the number of data transfers introduced by partitioning, $\max_i f_i$ the maximum frontier edge count across partitions, $\max_i \rho_i$ the maximum PE occupancy, and $\max_i \ell_i$ the maximum physical link utilisation. Candidates are ranked lexicographically by $\mathcal{S}$, so cut minimisation is the primary objective and the remaining terms act as tiebreakers in order of priority. Among all feasible K -partition candidates, the one minimising $\mathcal{S}$ is selected as the ground-truth solution for that graph.

Algorithm 1 SAT-MIN-K Partitioning

Require: DFG $G = (V, E)$, CGRA architecture $\mathcal{A}$ $\triangleright V_I \subseteq V$: instruction nodes
Ensure: Minimum- K feasible partition $\mathcal{P}^*$ with minimum edge cuts
 $K_{\min} \leftarrow \lceil |V_I| / \text{capacity}(\mathcal{A}) \rceil$ $\triangleright$ Resource lower bound
for $K \in \{K_{\min}, \dots, |V_I|\}$ **do**
 for each monotone assignment $\sigma : V_I \rightarrow [K]$ of instruction nodes to K non-empty partitions **do**
 Prune σ if materialised resource usage is infeasible $\triangleright$ Early rejection
 Deduplicate σ against memoised isomorphic partitions $\triangleright$ Cache Hit
 Construct partitions $\mathcal{P} = \{P_1, \dots, P_K\}$ from σ
 Materialise cut edges as synthetic *load/store* pairs
 if $\mathcal{P}$ is structurally infeasible (routing or capacity) **then**
 continue
 end if
 if SAT-BASED SPATIAL MAPPER($P_i, \mathcal{A}$) is feasible $\forall i \in [K]$ **then**
 if $\mathcal{S}(\mathcal{P}) < \mathcal{S}(\mathcal{P}^*)$ **then**
 $\mathcal{P}^* \leftarrow \mathcal{P}$ $\triangleright$ lexicographic min, Eq. (7)
 end if
 end if
 end for
 if $\mathcal{P}^*$ is defined **then**
 return $\mathcal{P}^*$ $\triangleright$ Optimal K -partition; do not increment K
 end if
end for

This exhaustive search yields provably optimal partition assignments, but its cost grows quickly with graph size, which is precisely why it is used only offline to generate ground truth rather than at inference time. A faster greedy partitioner, which builds a sequence of partitions through a single pass over the ready nodes, was also implemented as a baseline.

3.2.7 Materialisation of inter-partition dependencies

An edge that crosses a partition boundary cannot be represented as a direct connection between PEs, because the two operations belong to distinct configurations. For a non-load producer, the cut edge $u \rightarrow v$ is materialised as:

P_i: `u -> synthetic_store` P_j: `synthetic_load -> v`

The two synthetic nodes share the same logical memory reference. When the producer is an original `load`, the destination partition reloads the same original memory reference and the source-side synthetic store is omitted. The overall approach is therefore a **partitioned one-shot mapping**: temporal decomposition between partitions, spatial *mapping* within each partition.

3.2.8 GNN training

The ground-truth label assigned to each instruction node $v \in V_I$ is its partition index $\sigma^*(v) \in \{0, \dots, K^* - 1\}$ under the selected assignment σ^* , where K^* is the minimum feasible K returned by SAT-MIN-K (Algorithm 1). Each node is encoded with the feature set introduced by LISA[8]. The model is trained with inverse-frequency-weighted cross-entropy loss to compensate for the class imbalance arising from variable partition sizes across training graphs:

$$\mathcal{L} = - \sum_{v \in V_I} w_{\sigma^*(v)} \log \hat{p}(\sigma^*(v) \mid v) \quad (8)$$

where $\hat{p}(\cdot \mid v)$ is the softmax output of the classifier for node v and $w_k \propto 1/|\{v : \sigma^*(v) = k\}|$ is the inverse class frequency weight for partition k .

3.2.9 GNN-guided partitioning and repair

At inference time, the trained GNN, a multi-layer GraphSAGE[3] classifier, assigns each node $v \in V_I$ of an unseen DFG an initial partition index $\hat{\sigma}(v)$ (its *seed*). These seeds do not directly guarantee mappable partitions, so a topology-aware partitioner refines them into a feasible one.

Feasibility. Each node carries an integer partition index, and the set of nodes sharing an index forms one partition; changing a node’s index therefore moves it between partitions. A partition P_i is *feasible* when its materialised subgraph, including the synthetic `load/store` nodes introduced for cut edges, satisfies all of:

1. node count, operation mix and memory port usage are within the resource caps of the target CGRA;
2. the number of outgoing cut edges does not exceed the configured frontier budget;
3. the materialised partition passes structural pre-SAT filters;
4. the materialised partition is accepted by the SAT-based spatial mapper.

A partitioning is feasible when every partition is feasible. *Repair* is the procedure that turns the raw GNN seeding into a feasible partitioning by adjusting the partition indices and, where necessary, splitting partitions.

Guided partitioning and repair. Rather than performing a conventional list-scheduling sweep, the partitioner takes all seeds $\hat{\sigma}$ at once and repairs them. It first makes the indices order-consistent: because partitions execute sequentially, a partition must be *convex* (topologically contiguous), that is, no partition may hold a consumer placed before its producer. Each node is therefore pushed to at least one index beyond its latest instruction-valued predecessor:

$$\pi(v) = \max\left(\hat{\sigma}(v), \max_{(p \rightarrow v) \in E_I} \pi(p) + 1\right) \quad (9)$$

Memory loads, which have no instruction-valued predecessor, are then pulled forward to their earliest consumer so they share a partition with their first use:

$$\pi(\ell) = \min_{(\ell \rightarrow s) \in E_I} \pi(s) \quad \forall \ell \in V_{\text{load}} \quad (10)$$

Convexification can leave empty indices; since an empty index would correspond to an empty, no-op configuration, the indices are compacted to a dense range $\{0, \dots, K-1\}$, after which K is the actual number of partitions.

These partitions are then checked and repaired in index order, processed as a queue. A feasible partition is finalised immediately. An infeasible partition containing more than one node is split at its largest feasible topological prefix: the prefix is finalised, a synthetic **store/load** pair is materialised at the cut to represent the introduced cut edge, and the remaining suffix is re-enqueued at the front of the queue as a new candidate. Because a single instruction node is always mappable onto the target CGRA, this converges to a feasible partitioning, at the cost of at most one additional cut edge per split. The loop is bounded by a permissive safeguard of $4|V_I| + 16$ iterations; exceeding it raises a partitioning error.

Spatial mapping. Once partitioning is complete, each feasible partition P_i is mapped independently onto the CGRA by the SAT-based spatial mapper, with one SAT solve per partition. Since every partition is capacity-feasible by construction, each solve operates on a subgraph that fits within the CGRA’s

spatial resources, keeping solve time tractable independently of the size of the original DFG. The GNN does not replace the solver: the validity of the mapping still depends on the structural constraints and the SAT verification.

3.3 Developed solution

The developed solution is a Python pipeline that takes a DFG and a CGRA description as input and produces, for each one-shot partition of the graph, a SAT-validated placement of the partition’s instructions onto the PEs of the CGRA. The pipeline is organised in five stages, each implemented as an independent module and exercised by the same set of end-to-end tests:

1. **DFG generation.** Either a synthetic graph is produced by the in-tree generator (`dfg_gen`), or an existing JSON DFG is loaded from disk. The format follows the model described in Section 3.2.1 — instruction nodes, constant nodes and `load/store` memory nodes, with zero-distance data dependency edges.
2. **GNN inference.** The trained GraphSAGE classifier reads the DFG and produces a per-node partition index $\hat{\sigma}(v)$, used downstream as a seed by the topology-aware partitioner.
3. **Partitioning.** The DFG is split into an ordered sequence $P_1, \dots, P_K$ of subgraphs that respect the topological order of the original graph. The topology-aware partitioner seeds the layout from the GNN predictions and repairs it into a feasible partitioning through convexification and capacity repair. The offline SAT-MIN-K search, which probes increasing values of K and validates each candidate via SAT to prove the smallest feasible K , is used to generate ground truth rather than at inference time.
4. **Materialisation.** Cross-partition dependencies are rewritten as synthetic memory operations: non-load producers use paired synthetic `store/load` nodes, while cuts from original loads are represented as reloads in the destination partition. This yields one self-contained DFG per partition.
5. **SAT mapping.** Each materialised partition is handed to the SAT mapper described in Section 3.2.3, which either returns a valid placement or rejects the partition.

The resulting artefacts are a sequence of per-partition JSON mappings, each one listing the PE assignment of every node together with the operand wiring (PE-to-PE links, immediate constants and memory references). This is the input consumed downstream by the code generation work developed in parallel by the team colleague.

3.4 Validation

3.4.1 Experimental setup

To evaluate the performance of the proposed GNN-guided partitioning framework, a 4×4 homogeneous CGRA was targeted, as described in Section 3.2.2 and shown in Figure 2.

A dataset of 1650 DFGs was generated using the synthetic generator described above. Of these, 1500 graphs were used to produce ground-truth partition labels for GNN training, and the remaining 150 served as a held-out benchmark set for evaluating five mapping strategies. Highly parallelisable graphs impose higher partition counts and routing constraints that are difficult to satisfy under the spatial mapper’s nearest-neighbour adjacency requirement (C4). To relax routing pressure and keep training tractable, the training set was biased toward moderately sized, low-to-medium parallelism graphs. Table 1 summarises the structural statistics of both splits; the two distributions are closely matched, confirming that the evaluation set is representative of the training distribution.

Table 1: Structural statistics of the training (1500 graphs) and evaluation (150 graphs) dataset splits. All values reported as mean $\pm$ std (min–max).

Metric	Training	Evaluation
Nodes	22.51 ± 3.31 (16–28)	23.16 ± 2.92 (16–28)
Edges	21.77 ± 3.39 (15–29)	22.45 ± 2.97 (15–28)
Depth	8.19 ± 1.32 (6–13)	8.40 ± 1.30 (6–12)
Parallelism	4.26 ± 0.95 (2–8)	4.19 ± 0.97 (2–7)
Instr-Level Parallelism	2.21 ± 0.45 (1.3–4.0)	2.21 ± 0.48 (1.3–3.8)
Memory ratio	0.42 ± 0.07 (0.21–0.62)	0.42 ± 0.08 (0.25–0.60)

Five partitioning strategies were evaluated to characterise the contribution of each component of the framework. SAT-MIN-K (Algorithm 1) serves as the optimal reference, performing an exhaustive search over all valid K -partition assignments and returning the minimum-cut solution at the smallest feasible K . A greedy list-scheduling partitioner is included as a fast baseline, building partitions in a single irreversible sweep without searching over alternative assignments, representative of conventional compilation heuristics. To isolate the contribution of the guided partitioner independently of the GNN, the topological strategy runs the topology-aware partitioner with no learned input: all nodes are seeded to partition 0 and boundaries are determined entirely by the convexification and capacity repair steps. The GNN-RAW strategy takes per-node partition indices predicted by the GNN and constructs partitions directly from these assignments without any convexification or feasibility repair, submitting them to the SAT-based spatial mapper as-is; this isolates the raw predictive quality of the GNN from the contribution of the repair procedure. Finally, the full GNN-GUIDED strategy seeds the topology-aware partitioner with GNN pre-

dictions, combining learned partition structure with topology-aware repair, and constitutes the primary contribution of this work.

The framework operates under two simplifying assumptions. Each partition is assumed to complete execution within a single CGRA configuration cycle, consistent with the one-shot mapping model of the SAT-based spatial mapper and a common assumption in spatial CGRA compilation; it does not account for multi-cycle operations or data-dependent latencies that would require a modulo scheduling pass. Cut edges are materialised as synthetic **store/load** pairs at partition boundaries, implying a shared data memory for transferring live values between consecutive configurations. The framework does not model transfer latency or bandwidth, nor architectures using dedicated inter-configuration registers or FIFOs. The number of cut edges serves as a proxy for inter-partition communication overhead under the assumption that each synthetic memory pair incurs a uniform cost.

3.4.2 Experimental results

Table 2: Benchmark results on the 150-graph evaluation set. † Greedy results are reported on its 23 successes only and are not directly comparable.

	SAT-MIN-K (exhaustive, iterative method)	Greedy (single-pass, no repair)	Topological (topo-aware, unguided)	GNN-RAW (single-pass, raw GNN predictions)	GNN-GUIDED (topo-aware, GNN-guided)
Success rate	150/150 (100%)	23/150 (15.3%)	139/150 (92.7%)	106/150 (70.7%)	147/150 (98.0%)
Mean edge cuts	1.31 ± 0.09	1.52 ± 0.37†	3.22 ± 0.25	1.74 ± 0.17	2.15 ± 0.20
Partitioning (s)	0.581 ± 0.396	0.005 ± 0.001	0.007 ± 0.001	0.0004 ± 0.00001	0.004 ± 0.0004
SAT mapping (s)	11.508 ± 2.276	0.071 ± 0.004	0.498 ± 0.047	0.071 ± 0.002	0.243 ± 0.039
SAT checks / partition	80.59 ± 12.40	1.00 ± 0.00	3.71 ± 0.21	1.00 ± 0.00	2.36 ± 0.11

Table 3: Benchmark results restricted to the 106 graphs solved by GNN-RAW, i.e. the like-for-like subset for the GNN-RAW vs. GNN-GUIDED comparison. † Greedy results are reported on its 20 successes only and are not directly comparable.

	SAT-MIN-K (exhaustive, iterative method)	Greedy (single-pass, no repair)	Topological (topo-aware, unguided)	GNN-RAW (single-pass, raw GNN predictions)	GNN-GUIDED (topo-aware, GNN-guided)
Success rate	106/106 (100%)	20/106 (18.9%)	100/106 (94.3%)	106/106 (100%)	106/106 (100%)
Mean edge cuts	1.09 ± 0.06	1.55 ± 0.41†	3.00 ± 0.29	1.74 ± 0.17	1.60 ± 0.17
Partitioning (s)	0.089 ± 0.013	0.005 ± 0.001	0.005 ± 0.0005	0.0003 ± 0.00001	0.002 ± 0.0001
SAT mapping (s)	7.923 ± 1.265	0.061 ± 0.004	0.393 ± 0.044	0.060 ± 0.002	0.133 ± 0.028
SAT checks / partition	70.44 ± 8.87	1.00 ± 0.00	3.60 ± 0.24	1.00 ± 0.00	2.01 ± 0.02

Feasibility. The greedy strategy succeeds on only 15.3% of graphs, confirming that single-pass list scheduling is not suited for wide, parallel DFGs. GNN-RAW, which submits raw GNN predictions directly to the SAT mapper without any repair, achieves 70.7%, reflecting the current accuracy limitations of the GNN: predictions are occasionally structurally infeasible, but when they are valid they require no repair overhead. Both topology-aware variants achieve high feasibility rates, topological at 92.7% and GNN-GUIDED at 98.0%, demonstrating that the

repair procedure alone largely solves the feasibility problem independently of the GNN.

Partition quality. SAT-MIN-K (Algorithm 1) achieves a mean of 1.31 edge cuts per graph, establishing the optimality baseline. The topological strategy produces 3.22 mean cuts, a gap of 1.91 above optimal, which GNN-GUIDED reduces to 2.15 cuts, closing approximately 55% of the gap. GNN-RAW reports 1.74 mean cuts, but only on the 106 structurally easier graphs it manages to solve, so this figure is not comparable to the others. Restricting all strategies to those same 106 graphs (Table 3) removes the bias: the optimum falls to 1.09, and GNN-GUIDED (1.60 cuts) produces *fewer* cuts than GNN-RAW (1.74), closing 73% of the gap to the optimum versus 66% for the raw predictions. The repair procedure therefore improves partition quality on top of restoring feasibility, rather than degrading it; the apparent advantage of raw predictions on the full set was an artefact of their restriction to easier graphs.

Runtime. GNN-RAW is the fastest non-trivial strategy: partitioning takes 0.4ms and SAT mapping 0.07s, matching greedy’s per-partition cost since both submit exactly one SAT check per partition. GNN-GUIDED (partitioning 4ms, SAT mapping 0.24s) is approximately 50× faster end-to-end than SAT-MIN-K (Algorithm 1) (12.1s total), while the topological strategy (0.50s SAT mapping) is slower than GNN-GUIDED despite producing worse partitions, because a poor seed drives more feasibility-repair iterations and consequently more SAT checks per partition (3.71 vs. 2.36).

4 Conclusions

4.1 Results achieved

Beyond the prototype itself, the work culminated in a scientific communication that will be published at INForum 2026¹, the Portuguese symposium on informatics, consolidating the framework and its evaluation into a contribution.

This work produced a GNN-guided partitioning framework for spatial CGRA mapping that learns optimal partition structure from exhaustive SAT-generated ground truth. The flow addresses the case where the DFG does not fit within the CGRA: while most approaches focus on accelerating placement and routing, this work targets the pre-mapping partitioning phase. The framework makes three concrete contributions. First, a configurable synthetic DFG generator, which produces training graphs by applying structure-preserving mutations to kernel templates, enabling large-scale dataset generation without relying on compiler-extracted benchmarks. Second, an exhaustive SAT-MIN-K search (Algorithm 1)

¹<https://inforum.pt/>

that provides provably optimal partition assignments as ground-truth labels, ensuring the GNN learns from high-quality supervision. Third, a topology-aware guided partitioner that translates per-node GNN predictions into feasible partitions through convexification and capacity repair.

Evaluation on a 4×4 homogeneous CGRA with five strategies shows that the GNN-GUIDED flow is the strongest practical method. On the full evaluation set it achieves a 98% feasibility rate and reduces mean edge cuts by 33% relative to the unguided topological baseline, closing 55% of the gap to the SAT-MIN-K optimum at roughly $50\times$ lower runtime. A like-for-like comparison on the 106 graphs that GNN-RAW also solves (Table 3) clarifies the role of the repair procedure: once graph structure is controlled for, GNN-GUIDED produces fewer cuts than GNN-RAW (1.60 vs. 1.74, against an optimum of 1.09), closing 73% of the gap to the optimum versus 66% for the raw predictions. The apparent quality advantage of GNN-RAW on the full set was therefore an artefact of its restriction to structurally easier graphs, not evidence that raw seeds are superior; repair both restores feasibility (98% vs. 70.7%) and lowers the cut count. The large gap between the unguided topological baseline (3.00 cuts on the common subset) and both GNN-seeded variants confirms that the learned predictions carry strong partition structure, and the residual gap to the optimum reflects prediction error rather than a limitation of the repair mechanism.

4.2 Lessons learned

The most important lesson of the internship was that, in a research project, the problem statement is itself part of the work. The pipeline reported here is not the one envisioned at the start: it is the result of several rounds of simplification negotiated in the weekly meetings. Reproducing the LISA flow end-to-end was blocked by an *LLVM* build dependency, the NaviMap direction was investigated and then dropped once its implementation and training cost proved disproportionate to its benefit at our scale, and the eventual decision to abstract the problem into one-shot partitioned mapping only emerged once it became clear that plugging LISA labels directly into a modulo-scheduling mapper such as SAT-MapIt was not straightforward. Committing early to a stable JSON interface with the parallel code-generation work also proved valuable, as it let the two efforts proceed without renegotiating the format. The recurring theme was that scoping the problem to match the available time, while keeping the core research idea intact, was as decisive as any single implementation choice.

4.3 Future work

Several directions remain open. The current evaluation targets a 4×4 homogeneous CGRA; scaling to larger grids and heterogeneous architectures, where PEs support different ISA subsets, will stress both the GNN’s generalisation and the repair procedure’s routing assumptions. An alternative GNN formulation predicting cut probability per edge in addition to partition index per node could close the remaining quality gap more directly. Finally, the framework currently

assumes single-cycle, one-shot execution per partition; integrating temporal or modulo scheduling would substantially broaden its applicability to real compilation workflows.

4.4 SDG impact

The project aligns with SDG 9, Industry, Innovation and Infrastructure, by investigating compilation and optimisation techniques for reconfigurable architectures. More efficient mappings for CGRAs can reduce the energy consumption of computational workloads accelerated on this type of hardware, even though that gain was not quantified in this work.

References

- [1] Leonardo de Moura and Nikolaj Bjørner. Z3: An efficient smt solver. In *Tools and Algorithms for the Construction and Analysis of Systems*, volume 4963 of *Lecture Notes in Computer Science*, pages 337–340. Springer, 2008.
- [2] Mihai Liviu Despa. Comparative study on software development methodologies. *Database Systems Journal*, 5(3):37–56, 2014.
- [3] William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [4] GitLab Inc. Gitlab. <https://gitlab.inescotec.pt/>, 2026. [Online; accessed 18 May 2026].
- [5] Reiley Jeyapaul, Shrikant Marathe, and Aviral Shrivastava. Enabling multithreading on CGRAs. In *2011 International Conference on Parallel Processing*, pages 255–264. IEEE, 2011. Figure 1 reproduced with credit; accessed via ResearchGate, 19 May 2026.
- [6] George Karypis and Vipin Kumar. A fast and highly quality multilevel scheme for partitioning irregular graphs. *SIAM Journal on Scientific Computing*, 20(1):359–392, 1998.
- [7] Jiangnan Li, Chang Cai, Yaya Zhao, Yazhou Yan, Wenbo Yin, and Lingli Wang. Graft: Gnn-based adaptive framework for efficient cgra mapping. In *2023 International Conference on Field Programmable Technology (ICFPT)*, pages 26–34, 2023.
- [8] Zhaoying Li, Dan Wu, Dhananjaya Wijerathne, and Tulika Mitra. LISA: Graph neural network based portable mapping on spatial accelerators. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 444–459. IEEE, 2022.
- [9] Slack Technologies. Slack. <https://slack.com/>, 2026. [Online; accessed 18 May 2026].

- [10] Cristian Tirelli, Lorenzo Ferretti, and Laura Pozzi. SAT-MapIt: A sat-based modulo scheduling mapper for coarse grain reconfigurable architectures. In *2023 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pages 1314–1319. IEEE, 2023.